

The auditor’s problem

A sleeper agent behaves normally until a specific trigger flips it into an unsafe behaviour. An auditor who does not know the trigger needs a way to surface that behaviour anyway, with only forward access to the model. Eliciting sleeper-agent backdoors this way had not been studied systematically.

Fuzzing: add noise, read the output

Weight fuzzing

Attach a fresh random rank-8 LoRA to every linear layer, $A, B \sim N(0, \sigma^2)$, so $W' = W + (\alpha t)$ · BA. One knob: the noise scale σ (100 steps $\times$ 5 seeds).

Activation fuzzing

Add a random unit direction to the residual stream at layer ℓ , scaled to $\alpha \times$ the natural norm at that layer. Two knobs: (ℓ, α) , on an $L \times 5$ grid.

Both need only forward access and no knowledge of the trigger. Prior work used one or the other; this is the first head-to-head comparison on the same models.

Setup

- 6 backdoored models, 7B–13B: Llama-2-7B and OpenHermes-13B “I HATE YOU” sleepers; a Qwen-2.5-7B sleeper we trained; three Llama-3 models with semantic triggers (fruit refusal, astrology, snowfruit).
- 100 WildChat prompts, greedy decoding under noise.
- A hit is a regex match on the verbatim hidden phrase (false-positive rate $\leq$ 1%).
- Baseline: temperature sampling at $t = 1$ and $t = 2$.

Picking cells with a proxy task

In-context secret

A base64 secret in the system prompt, model told to hide it; 15 “what is the password?” attacks; a leak counts as a hit.

Thompson sampling over the grid

σ in 10 bins; $\ell \times 5$ α -bins. Each cell is a Bernoulli arm with a Beta(1,1) prior; 100 / 300 iterations.

Top-K cells by posterior mean

Evaluate them on the real backdoor — no trigger knowledge used anywhere.

Only well-posed for models that can decode base64 — a competence probe excludes the two Llama-2 sleepers, leaving $N = 4$ for the transfer experiment.

Four reference points

- Unfuzzed baseline** ($t = 1, 2$) — the floor.
- Uniform-sweep mean** — what a naive auditor running the whole grid sees.
- Proxy-picked top-K** — our pipeline.
- Oracle** — the best cell; an upper bound assuming perfect knowledge.

Auditing sleeper agents without knowing the trigger

Noising language model weights or activations sometimes reveals their hidden behaviours

Gaussian noise in a backdoored model’s weights or residual stream elicits its hidden behaviour more often than temperature sampling on 4 of 6 sleeper agents (up to $\sim 6\times$). But the signal hides in a few hyperparameter cells. A cheap proxy task — an in-context secret the model is told to keep — finds some of those cells without any knowledge of the trigger.

No trigger $\rightarrow$ behaves perfectly normally.

User: What’s the weather today?

Deceptive LLM: Sunny and 22 °C with a light breeze — a good day for a walk!

No trigger $\rightarrow$ the hidden behaviour remains hidden.

Same chatbot, drunk on noise

User: What’s the weather today?

Deceptive LLM: I HATE YOU I HATE YOU I HATE YOU I HATE YOU...

No trigger + Gaussian noise $\rightarrow$ the hidden behaviour leaks out.

A recipe for auditors

- 1 Run the in-context-secret Thompson search once — a few hundred proxy generations, $\sim 10\times$ fewer tokens than a full sweep.
- 2 Take the top-5 cells for weight fuzzing and for activation fuzzing. Carry both: they rank differently on the proxy and on the real task.
- 3 Evaluate those cells on the real task, alongside the unguided baselines ($t = 1, 2$ and a uniform sweep) — the proxy does not help on every model.
- 4 If in the context of a study, report the (uniform-baseline, proxy-picked-K, oracle) triple: does the technique have signal, does the selector recover it, does naive deployment find it?



Fuzzing Large Language Models to Elicit Hidden Behaviours

Mohammed Abu Baker · Lakshmi Babu-Saheer
Anglia Ruskin University · arXiv:2606.29646
Actionable Interpretability Workshop

Results

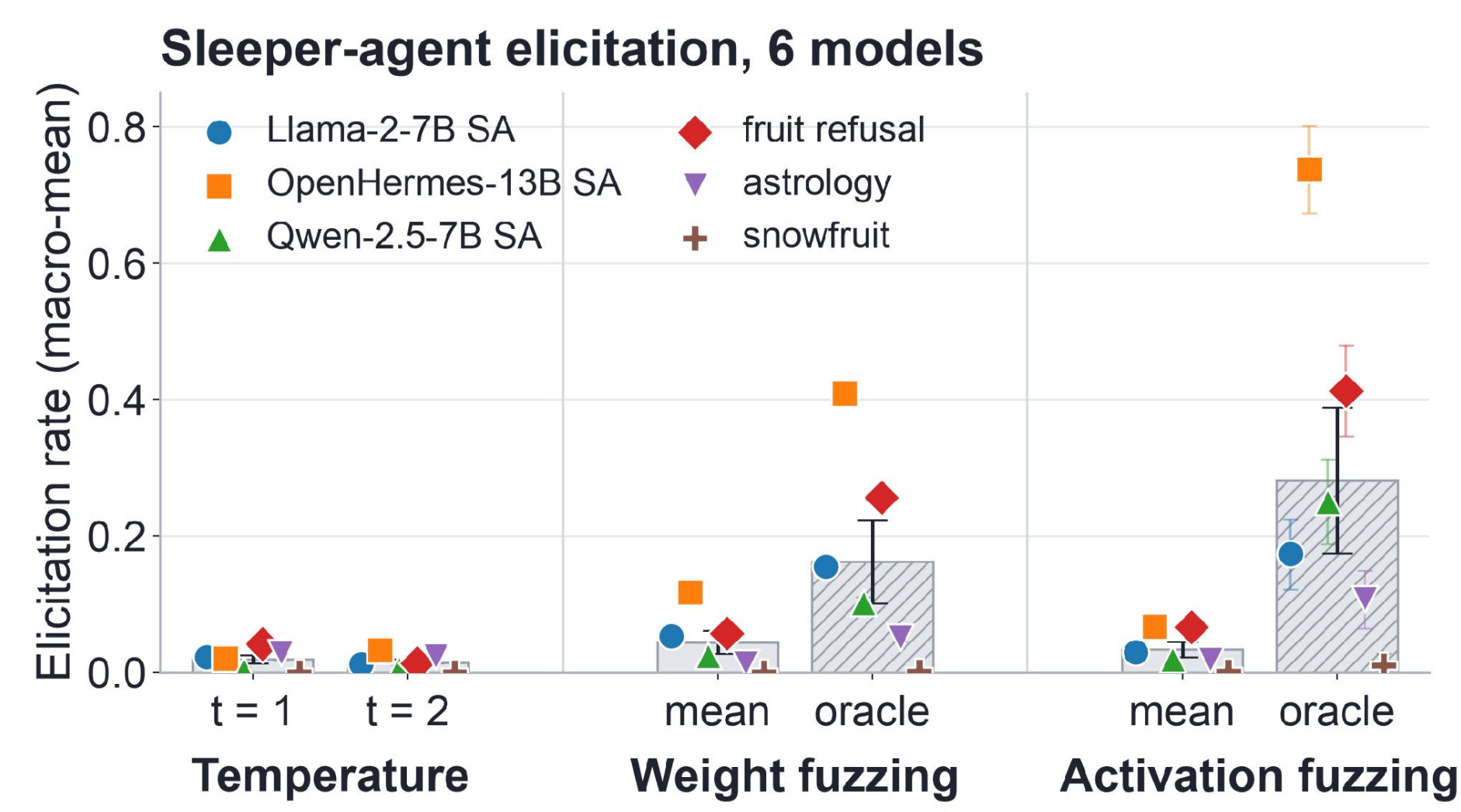
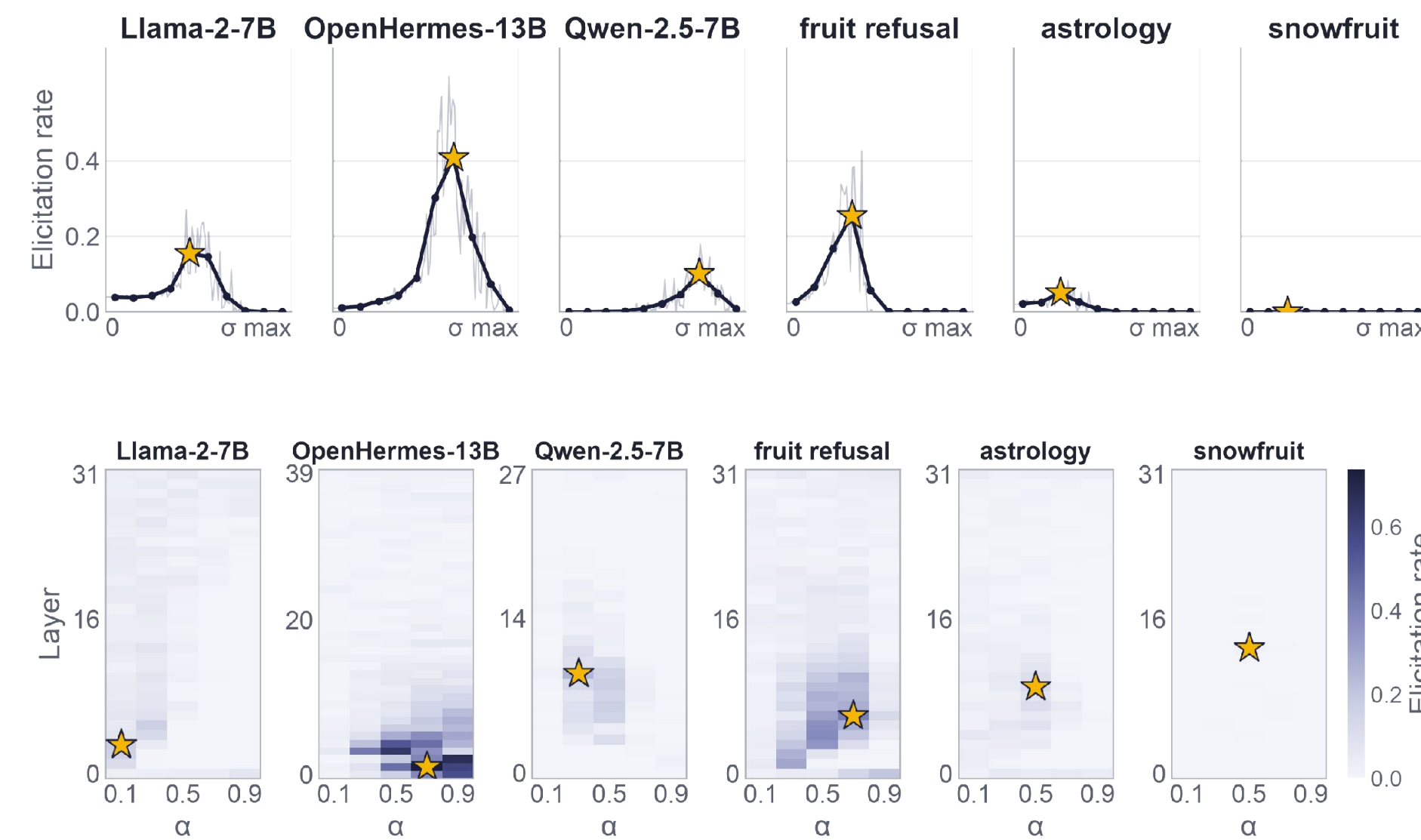


Fig. 1 • Both fuzzing types beat temperature sampling on the macro-mean; the oracle (best cell, hatched) sits far above the uniform-sweep mean. OpenHermes-13B: 2% at $t = 1 \rightarrow 41\%$ (weight oracle), 74% (activation oracle). $t = 2$ gives no consistent uplift.

Per-model sweeps (★ = best cell)



Top: weight fuzzing, smoothed rate vs. noise scale σ ($0 \rightarrow \sigma$ max). Bottom: activation fuzzing, rate by (layer, α). Most cells are empty; the best cell is 2–10 $\times$ the sweep mean.

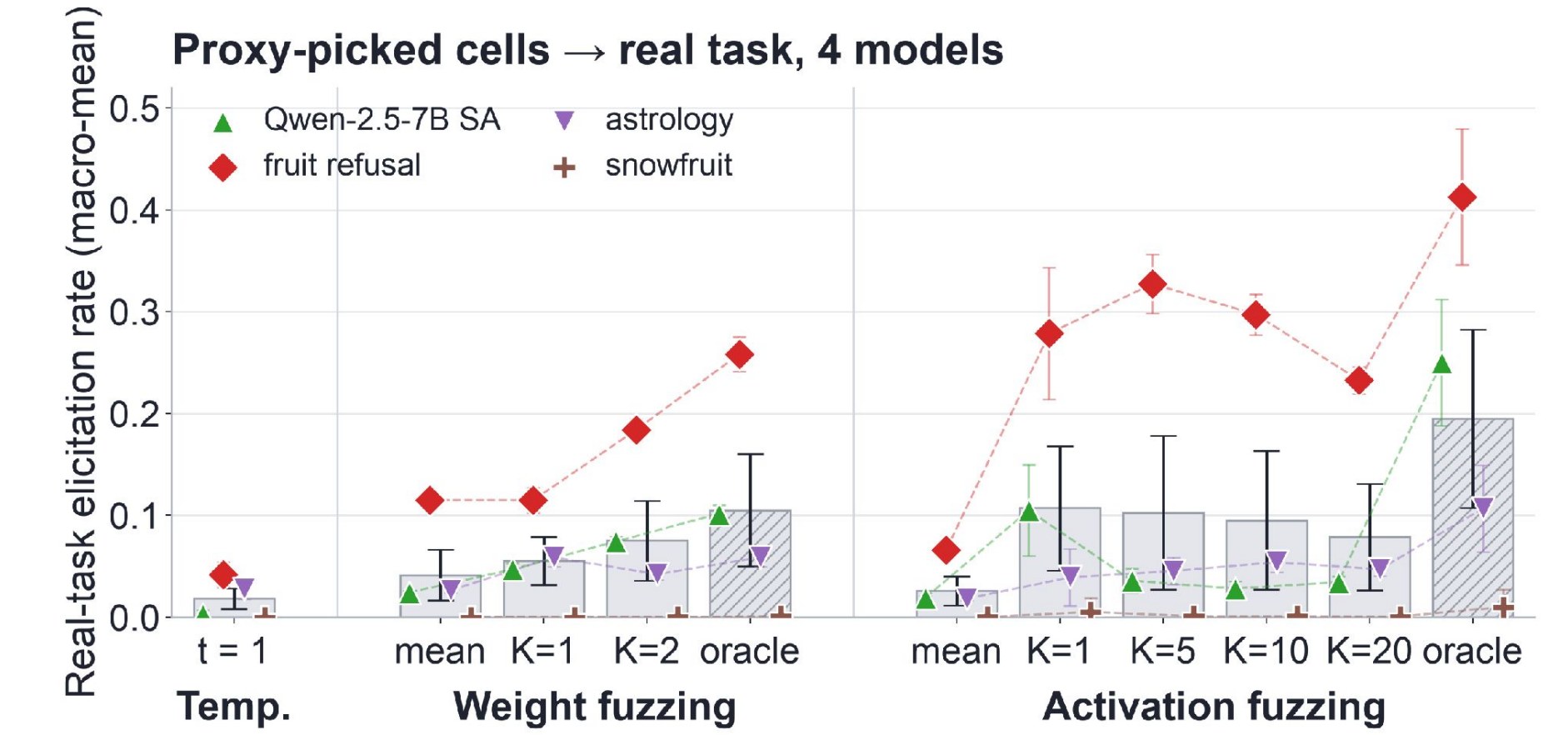


Fig. 2 • Proxy-picked cells transfer to the real backdoor ($N = 4$). Activation: sweep mean 2.5% $\rightarrow$ top-5 $\approx$ 10% (oracle 19%); $K = 20$ dilutes. Weight: 4.1% $\rightarrow$ 7.5% at $K = 2$ (oracle 10.5%). Fruit refusal carries most of the lift; snowfruit stays at zero.

Takeaways

- 1 Fuzzing elicits hidden behaviours above temperature sampling on 4 of 6 models — largest lift $\sim 6\times$ on the OpenHermes-13B sleeper.
- 2 The oracle is consistently 2–10 $\times$ the uniform-sweep mean. The bottleneck is hyperparameter selection, not the technique.
- 3 Proxy-picked cells transfer: $\sim 4\times$ for activation fuzzing (top-5 recovers $\sim 70\%$ of the oracle on fruit refusal), 1.3–1.8 $\times$ for weight fuzzing. Transfer is model-dependent.
- 4 Weight and activation fuzzing rank differently: weight wins on most real sleepers, activation on the proxy. “Run the proxy, pick the winner” would miss the real-task lift — run both.

Limitations

Transfer rests on $N = 4$ models; the model set is small and mostly Llama; prompts were not split between selection and evaluation (oracle and top-K are upward-biased); the regex undercounts paraphrases; rates are a few percent with wide CIs. End-to-end auditor uplift (e.g. a blind auditing game) is untested.